

Enumereringar

Ibland kanske man vill använda någon form av namngivna konstanter för att representera något, t.ex. färger. Vi kan då t.ex. deklarerar ett antal variabler av typen `int` och ge dem unika värden. Sedan kan vi använda dessa variabler för att t.ex. indexera en vektor med färger. Låter klumpigt, eller hur? Det finns ett lättare sätt att åstadkomma följder av tal i C++, nämligen med *enumereringar*. Allmänt definieras en enumerering på följande sätt:

```
enum namn { element1, ..., elementN };
```

En enumerering innehåller en mängd element, som kommer att enumerera börjande från 0. På så vis associeras varje element med ett unikt värde. En enumerering ges även ett namn, så att man kan skapa variabler med enumereringen som datatyp. Ett exempel på en enumererad regnbåge:

```
enum Color { Black, White, Red, Orange, Yellow, Green, Blue, Purple, Indigo };
```

På engelska för omväxlings skull. Här har vi nu definierat en ny datatyp `Colors`, och vi kan deklarerar variabler av denna typ om vi vill:

```
Color EnFarg;  
Color Regnbage [5];
```

Denna datatyp är dock speciell, i och med att variabler av en enumerad datatyp endast kan innehålla de värden som gavs vid enumereringens definition. Så i vårt exempel kan `EnFarg` ges alla värden från `Black` till `Indigo`. En variabel tilldelas ett värde helt normalt:

```
Farg = Red;  
Regnbage [0] = Red;  
Regnbage [1] = Orange;
```

De enda operationer som är tillåtna med enumereringar är tilldelning och jämförelse. Man kan t.ex. inte addera eller subtrahera enumereringar. Man kan även endast tilldela enumererade variabler värden som finns i enumereringen. Följande är alltså inte tillåtet:

```
Color Farg;  
Farg = 10;    // fel datatyp  
Farg = Gray; // ingen sådan färg i enumereringen
```

Man kan tilldela en enumerering en `int` om man först konverterar den till en enumerering:

```
Color Farg;  
Farg = (Color)3; // ok, blir 'Orange'  
Farg = (Color)8; // ok, blir 'Indigo'  
Farg = (Color)45; // fel, ingen sådan färg
```

Det sista exemplet är illegalt, eftersom det inte finns så många värden i enumereringen. Som redan tidigare nämnts så är en enumerering i grund och botten en `int`, därför är ovanstående konvertering möjlig. Man kan ge enumereringar andra värden än sådana som börjar från 0 och ökar med 1 per värde. Det görs genom att skriva värdet efter enumereringselementet. Om man t.ex. vill börja en

serie från 10 kan man göra följande:

```
enum Bilar { Opel = 10, BMW, Mercedes, Ford };
```

Nu får `BMW` värdet 11, `Mercedes` värdet 12 och `Ford` värdet 13. Man kan även ge varje element ett unikt värde, men de bör vara i stigande ordning när man går högerut. Man kan även ge flera element samma värde om det finns behov för detta.

```
enum KursBitar { Forelasning = 1, Tent = 2, RO = 4, Kompendium = 8, Studerande =  
enum Error { Critical = 0, Fatal = 0, Serious, Warning, Ok };
```

Ovan har vi skapat en *binär* enumerering vars värden kan kombineras med t.ex. `or` för att på så vis kombinera flera flaggor i ett enda `int`-värde. Användningsområdena för enumereringar är mycket varierande. Ofta används de där man vill ha en egen datatyp som kan användas som en `int` (t.ex. för att indexera vektorer), eller där man vill ha en datatyp som endast kan ha ett fåtal fördefinierade värden.

[Förutgående](#)

Sammansatta datatyper

[Hem](#)

[Upp](#)

[Nästa](#)

Definiera egna datatyper